



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA
**Dipartimento di Informatica, Sistemistica e
Comunicazione**
Corso di Laurea in Informatica

Integrazione del chatbot aziendale su Microsoft Teams e WhatsApp Business

Relatore: Prof.ssa Sara Lucia Manzoni
Tutor: Sandro Zampieri

Tesi di Laurea di:
Nicholas Guido Boidi
Matricola 894417

Anno Accademico 2024-2025

Ringraziamenti

Vorrei ringraziare l'azienda Par-Tec S.p.A. per l'opportunità di stage formativo. In particolare ringrazio il mio tutor aziendale, Sandro Zampieri, che ha riposto fiducia in me fin dall'inizio e mi ha permesso di imparare molto del contesto aziendale. Ringrazio inoltre i miei colleghi aziendali che hanno reso l'ambiente lavorativo piacevole e stimolante, si sono resi disponibili a trasferirmi le loro conoscenze e mi hanno offerto aiuto a svolgere e concludere il lavoro.

Un ringraziamento speciale va anche alla mia relatrice, professoressa Sara Lucia Manzoni, per il prezioso supporto, i consigli e la disponibilità offerta durante la stesura della tesi.

Ringrazio i miei compagni di università. Senza di loro non sarei mai riuscito ad arrivare dove sono ora. Vi ringrazio per tutti i momenti passati insieme, per le risate, le chiacchierate, le pause pranzo, il supporto nei momenti difficili, lo studio e le bevute insieme. Ringrazio la mia amica Vale per tutte quelle volte in cui mi ha aiutato a studiare esami difficili e per tutti gli appunti che mi ha gentilmente passato. Ringrazio i miei amici Donghi, Nadim, Vale e Pollo con cui ho svolto questo percorso insieme e con cui ho condiviso successi e fallimenti. Ringrazio i miei amici conosciuti durante l'erasmus, per aver reso quell'esperienza un'avventura indimenticabile.

Ringrazio mio padre Pietro, mia madre Corinna e mia sorella Alexandra per il supporto costante durante questi tre anni e per avere creduto da sempre in me. Vi ringrazio per la pazienza avuta nei momenti difficili, per le parole di incoraggiamento e l'aiuto offerto.

Ringrazio me stesso per non avere mai mollato, per avere creduto in me e per essere andato avanti anche nei momenti difficili. Sono sicuro che il futuro mi ricompenserà per tutti gli sforzi che mi hanno permesso di arrivare dove sono ora.

Abstract

Nel contesto delle automazioni dei processi aziendali, i chatbot stanno assumendo un ruolo di rilevanza nella gestione delle comunicazioni interne ed esterne. Questa tesi descrive il lavoro di stage curriculare svolto presso Par-tec S.p.A. finalizzato allo sviluppo di uno strato di middleware che permette al chatbot di comunicare con canali di comunicazione esterni, quali Microsoft Teams, Slack e WhatsApp Business. L'obiettivo principale è facilitare l'interazione tra il chatbot e l'utente finale, che può essere sia il dipendente che il cliente dell'azienda, attraverso una piattaforma di comunicazione familiare e facilmente accessibile. In particolare l'attività ha previsto l'analisi dei requisiti e dell'architettura esistente, la progettazione del middleware, lo sviluppo tecnico delle integrazioni tramite RESTful API e il testing delle soluzioni ottenute. I risultati hanno confermato la fattibilità tecnica del progetto e il suo valore aggiunto all'azienda. La soluzione fornita apre la strada ad interessanti prospettive in ambito enterprise. La tesi si conclude con una riflessione sui vantaggi ottenuti, sulle competenze acquisite durante lo stage e sulle prospettive oltre che possibili sviluppi offerti dal sistema implementato.

English Abstract

In the context of business process automation, chatbots are assuming an important role in the management of internal and external communication. This thesis describes the curricular internship work carried out at Par-tec S.p.A. aimed at developing a middleware layer that enables the chatbot to communicate with external communication channels, such as Microsoft Teams, Slack and WhatsApp Business. The main objective is to facilitate interaction between the chatbot and the end user, who can be either an employee or a customer of the company, through a familiar and easily accessible communication platform. In particular, the activity involved the analysis of the requirements and the existing architecture, the design of the middleware, the technical development of the integrations via RESTful API and the testing of the solutions obtained. The results confirmed the technical feasibility of the project and its added value to the company. The solution provided opens the way to interesting prospects in the enterprise environment. The thesis concludes with a reflection on the benefits obtained, the skills acquired during the internship and the prospects as well as possible developments offered by the implemented system.

Indice

Ringraziamenti	1
1 Introduzione	5
2 Stato dell'arte	6
2.1 Microsoft Bot Framework SDK	6
2.2 Problemi con le REST API	7
2.3 Definizione e funzionamento dei chatbot	7
2.4 Applicazioni aziendali dei chatbot	8
2.5 Le piattaforme di messaggistica	9
2.5.1 Microsoft Teams	10
2.5.2 WhatsApp Business Platform	10
3 Architettura iniziale e tecnologie utilizzate	11
3.1 L'Azienda: Par-Tec S.p.A.	11
3.2 Architettura del chatbot aziendale	12
4 Integrazione con Microsoft Teams	14
4.1 Analisi dei requisiti	15
4.2 Progettazione dell'integrazione	16
4.3 Sviluppo e test	18
4.3.1 Gestione del flusso attraverso il controller	18
4.3.2 Gestione della sicurezza e dell'autenticazione	19
4.3.3 Gestione dello stato conversazionale con Redis	21
4.3.4 Creazione di risposte interattive per Teams	22
4.3.5 Processo di testing	24
4.4 Risultati ottenuti	25
5 Integrazione con Slack	27
5.1 Il ruolo del Microsoft Bot Framework	27
5.2 Configurazione del canale Slack	28

Indice

5.3	Risultati e validazione architetturale	29
6	Integrazione con WhatsApp Business	30
6.1	Analisi e scelta della piattaforma	30
6.2	Sfide di integrazione e sviluppo	31
6.3	Risultati ottenuti	33
7	Conclusioni	35
7.1	Sintesi dei risultati raggiunti	35
7.2	Valore aggiunto per l'azienda	36
7.3	Competenze acquisite durante lo stage	37
7.4	Prospettive future e possibili sviluppi	38

Capitolo 1

Introduzione

Negli ultimi anni, l'utilizzo dei chatbot si è affermato come una delle soluzioni tecnologiche più usate nel mondo della comunicazione aziendale. In particolare le aziende stanno progressivamente adattando sistemi di intelligenza artificiale per gestire e automatizzare l'interazione tra cliente e dipendente, riducendo i tempi di risposta e semplificando l'accesso ai servizi.

In questo contesto si inserisce l'esperienza di stage curriculare svolta presso **Par-Tec S.p.A.**, azienda attiva nel settore dei servizi IT, con l'obiettivo di integrare il chatbot aziendale già esistente all'interno delle piattaforme di comunicazione **Microsoft Teams**, **Slack** e **WhatsApp Business**.

Il progetto ha previsto un'analisi iniziale dell'architettura del chatbot già sviluppato dall'azienda, la progettazione e lo sviluppo dell'integrazione su Microsoft Teams attraverso l'utilizzo del *Microsoft Bot Framework* e delle *RESTful API* fornite dal Framework, oltre alla realizzazione di test funzionali e all'ottimizzazione del sistema. Successivamente, è stata condotta un'indagine tecnica e di mercato per valutare la fattibilità dell'integrazione con WhatsApp, tenendo conto delle API disponibili, dei requisiti di sicurezza e delle possibili applicazioni future. Terminata l'analisi tecnica, è iniziata l'implementazione del layer per WhatsApp, utilizzando i servizi forniti da *Azure Portal* di proprietà di Microsoft Teams.

Questa tesi ha l'obiettivo di documentare il lavoro svolto, evidenziando le scelte progettuali, le soluzioni tecniche adottate, i risultati raggiunti e le prospettive di evoluzione del sistema chatbot orientato su una comunicazione multicanale e scalabile.

Capitolo 2

Stato dell'arte

2.1 Microsoft Bot Framework SDK

Il Microsoft Bot Framework SDK è uno degli strumenti più diffusi per lo sviluppo di chatbot aziendali, che fornisce una serie di librerie e strumenti per creare, testare e distribuire chatbot su diverse piattaforme. L'SDK supporta numerosi linguaggi di programmazione, tra cui C# e Node.js, e facilita la gestione dei flussi conversazionali, l'autenticazione e la gestione dei canali di comunicazione (Teams, Slack, Facebook Messenger, ecc.) [5].

Tuttavia, nonostante l'SDK offra una serie di funzionalità preconfigurate, ho scelto di non utilizzare l'SDK nel progetto della tesi, per le seguenti ragioni:

- **Flessibilità e controllo:** L'SDK, pur offrendo strumenti pronti all'uso, ha una struttura che può risultare troppo rigida in alcuni casi. Avendo bisogno di una gestione più diretta delle richieste API, ho preferito lavorare a livello di RESTful API, che mi ha permesso di avere un maggiore controllo sulla gestione delle comunicazioni tra il chatbot e i canali di comunicazione esterni.
- **Integrazione con sistemi esistenti:** Un altro motivo per cui non ho utilizzato l'SDK è che il mio progetto è basato su NestJS, un framework Node.js già utilizzato dall'azienda. Scrivere codice direttamente su NestJS mi ha permesso di integrare meglio il chatbot con il resto dell'architettura aziendale e di mantenere la coerenza con il progetto esistente. Le REST API mi hanno permesso di interagire facilmente con il sistema, senza dover adattare il codice dell'SDK alle specifiche necessità aziendali.
- **Scalabilità:** Anche se l'SDK semplifica l'integrazione con canali già supportati, l'uso delle REST API offre una maggiore flessibilità e per-

sonalizzazione per canali non direttamente supportati dall'SDK o per esigenze aziendali specifiche.

2.2 Problemi con le REST API

L'utilizzo delle API, in particolare per il canale WhatsApp Business, non è sempre stato semplice. La documentazione ufficiale per l'uso delle REST API di WhatsApp, sebbene disponibile, non è sempre esaustiva o facile da seguire. In particolare, alcuni argomenti, come l'autenticazione, l'uso dei templates o messaggi interattivi, sono spesso trattati in modo non esaustivo, con esempi non sempre chiari o completi. Questo ha reso difficile non solo implementare la logica necessaria, ma anche risolvere problemi pratici legati all'uso delle API, come la gestione delle risposte o la corretta gestione degli errori.

Soprattutto, la documentazione online del servizio di Azure che permette di collegare WhatsApp Business al proprio bot manca di concetti fondamentali come l'autenticazione delle richieste o la struttura del corpo. È stato quindi necessario eseguire molteplici esperimenti o prove e correggere errori per comprendere a pieno il comportamento delle API.

Questo ha reso il lavoro di integrazione più complicato e lungo rispetto ad altri canali più documentati, come Microsoft Teams e Slack, dove le API e la documentazione sono più stabili e complete. La mancanza di esempi pratici e guide complete per il canale WhatsApp ha quindi contribuito a un certo ritardo nello sviluppo.

Tuttavia, anche l'implementazione del chatbot su Microsoft Teams ha richiesto un tempo considerevole, in quanto era la prima volta che lavoravo con questa piattaforma. La familiarità con il framework e la necessità di adattarlo alle esigenze specifiche dei canali di comunicazione hanno contribuito a un processo di sviluppo più lungo del previsto. Nonostante ciò, l'esperienza acquisita ha permesso di gestire meglio il resto del progetto e mi ha permesso di approfondire la conoscenza delle tecnologie e degli strumenti coinvolti.

2.3 Definizione e funzionamento dei chatbot

Un chatbot è un software che simula ed elabora le conversazioni umane (scritte o parlate), consentendo agli utenti di interagire con i dispositivi digitali come se stessero comunicando con una persona reale. Può essere *rule-based*,

ossia basato su regole predefinite, oppure *AI-based*, sfruttando tecniche di elaborazione del linguaggio naturale e intelligenza artificiale [1]. Il chatbot aziendale oggetto della tesi è di tipo AI-based. Ovvero sfrutta la tecnologia del modello di LLM di ChatGPT per rispondere alle query fornite.

Un modello AI-based per i chatbot utilizza algoritmi avanzati di elaborazione del linguaggio naturale (NLP - Natural Language Processing) e apprendimento automatico (machine learning) per interpretare e comprendere le domande degli utenti e generare risposte in modo più naturale rispetto ai chatbot rule-based. A differenza dei chatbot tradizionali, che seguono un flusso di conversazione predefinito e rispondono solo a domande specifiche per cui sono stati programmati, i chatbot basati su AI generativa possono fare un ulteriore passo in avanti generando nuovi contenuti come output. Questi nuovi contenuti possono includere testo, immagini e suoni di alta qualità in base agli LLM su cui sono addestrati. Le interfacce chatbot con AI generativa sono in grado di riconoscere, con l'abilità di riassumere e tradurre secondo le esigenze, e creare contenuti in risposta alla domanda di un utente senza alcuna interazione umana [2].

2.4 Applicazioni aziendali dei chatbot

I chatbot stanno assumendo un ruolo sempre più centrale nell'automazione dei processi aziendali, in particolare per la gestione delle comunicazioni interne ed esterne. Le aziende si avvalgono di sistemi basati su intelligenza artificiale per semplificare e automatizzare l'interazione con clienti e dipendenti, con l'obiettivo di ridurre i tempi di risposta e di semplificare l'accesso a informazioni e servizi.

Le applicazioni dei chatbot in ambito aziendale possono essere suddivise in due macro-categorie principali: l'interazione con i clienti (uso esterno) e l'automazione dei processi interni e l'interazione con i dipendenti (uso interno).

Applicazioni esterne (verso i clienti)

- **Servizio clienti:** Offrire supporto continuato (24/7), rispondendo alle domande frequenti (FAQ - Frequently Asked Question), gestendo ri-

chieste di primo livello e smistando gli argomenti più complessi verso gli operatori umani.

- **Supporto commerciale:** Assistere gli utenti nella ricerca dei prodotti e guidarli durante la fase di acquisto.
- **Marketing e comunicazione:** Interagire con gli utenti tramite campagne mirate, realizzare sondaggi e raccogliere feedback per migliorare i servizi.

Applicazioni interne (verso i dipendenti)

- **Supporto risorse umane (HR):** Eseguire un'iniziale screening dei curriculum dei candidati e fornire risposte a domande dei dipendenti.
- **Help Desk IT:** Automatizzare la risoluzione di problemi tecnici comuni, assistere nella gestione dei sistemi IT e facilitare l'apertura di ticket di supporto.
- **Automazione di processi:** Semplificare attività come ad esempio aiutando un dipendente a identificare rapidamente il prodotto più adatto per un cliente.

Il progetto che è oggetto di questa tesi si identifica in questo contesto. Lo scopo è quello di facilitare l'interazione tra il chatbot aziendale di Par-Tec S.p.A. e l'utente finale, che può essere sia un dipendente sia un cliente dell'azienda. L'obiettivo primario è massimizzare l'utilità e il ritorno sull'investimento di questi strumenti che si ottiene migliorando l'accessibilità ai servizi aziendali con l'integrazione del chatbot all'interno di piattaforme di messaggistica familiari e di uso quotidiano.

2.5 Le piattaforme di messaggistica

La scelta dei canali di comunicazione su cui integrare un chatbot è un fattore cruciale per il successo del progetto, in quanto determina dove e come gli utenti finali interagiranno con il servizio. Per questo lavoro di tesi sono state selezionate due piattaforme leader nei rispettivi ambiti di applicazione: Microsoft Teams per la comunicazione aziendale interna e WhatsApp Business per l'interazione con i clienti esterni.

2.5.1 Microsoft Teams

Microsoft Teams è una piattaforma di comunicazione e collaborazione unificata, parte integrante della suite di prodotti Microsoft 365. Nata come strumento per la comunicazione interna alle aziende, unisce in un'unica applicazione funzionalità di chat persistente, videoconferenze, archiviazione e condivisione di file, e una profonda integrazione con le altre applicazioni dell'ecosistema Microsoft. La sua vasta adozione in contesti corporate lo rende un canale strategico per il deployment di chatbot e assistenti virtuali volti a supportare i dipendenti, automatizzare i processi interni e migliorare l'efficienza operativa.

2.5.2 WhatsApp Business Platform

WhatsApp Business è la versione dell'omonima applicazione di messaggistica istantanea, di proprietà di Meta, pensata specificamente per le aziende. Propone in due offerte principali: l'app gratuita "WhatsApp Business" per le piccole imprese e la "WhatsApp Business Platform", un'infrastruttura accessibile tramite API e rivolta ad aziende di medie e grandi dimensioni che necessitano di gestire un alto volume di conversazioni e di integrare il canale con i propri sistemi software, come CRM (Customer Relationship Management) o, in questo caso, un chatbot. Data il suo utilizzo a livello globale, rappresenta il canale per eccellenza per la comunicazione B2C (Business to Consumer), permettendo alle aziende di interagire con i propri clienti sulla piattaforma che già usano quotidianamente.

Capitolo 3

Architettura iniziale e tecnologie utilizzate

3.1 L'Azienda: Par-Tec S.p.A.

L'attività di stage descritta in questa tesi si è svolta presso Par-Tec S.p.A., un'azienda che da oltre 25 anni opera come fornitore e innovatore nel settore dei servizi IT in Italia e nel mondo. La mission aziendale consiste nell'aiutare le imprese a rispondere con efficacia e rapidità alle sfide del mercato, attraverso la fornitura di soluzioni e servizi IT di qualità, innovativi e personalizzati. L'offerta di Par-Tec si articola principalmente in tre aree:

- **Soluzioni applicative e infrastrutturali:** Progettazione e realizzazione di soluzioni personalizzate, basate sia su prodotti di mercato sia proprietari.
- **Servizi professionali qualificati:** Erogazione di servizi gestiti (Managed Services), Outsourcing, Help Desk 24/7 e Business Continuity.
- **Formazione:** Erogazione di percorsi formativi di alto profilo.

Il progetto di tesi si inserisce coerentemente in questo contesto, focalizzandosi sull'abilitare la comunicazione con il chatbot aziendale esistente attraverso canali esterni come Microsoft Teams, Slack e WhatsApp Business. L'obiettivo è potenziare questo strumento, rendendolo accessibile sulle piattaforme di messaggistica di uso quotidiano con l'obiettivo di massimizzarne l'adozione e l'utilità. L'integrazione rappresenta un'applicazione pratica della mission di Par-Tec, che mira a fornire soluzioni innovative per migliorare l'efficienza operativa e i processi di comunicazione aziendali [7].

3.2 Architettura del chatbot aziendale

L'architettura di partenza era costituita da un chatbot aziendale AI-based già sviluppato e funzionante per conto dell'agenzia di Tour Operator "Quality Group", cliente dell'azienda. Questo chatbot sfrutta un modello linguistico di grandi dimensioni (LLM), basato sulla tecnologia di ChatGPT, per analizzare e rispondere alle richieste degli utenti. La principale limitazione di questa architettura iniziale era l'assenza di un componente dedicato a gestire la comunicazione con canali di messaggistica esterni.

Il lavoro svolto si è quindi concentrato sulla progettazione e lo sviluppo di uno strato di middleware, un componente software intermedio il cui scopo è proprio quello di fare da "ponte" tra il nucleo del chatbot e le piattaforme di comunicazione esterne come Microsoft Teams, Slack e WhatsApp Business. La scelta è stata presa per garantire congruenza con l'infrastruttura tecnologica su cui è basato il progetto, ovvero il framework NestJS su ambiente Node.js.

L'assistente conversazionale oggetto di questo progetto di tesi è stato sviluppato per un primario tour operator italiano e la sua architettura è progettata per superare i limiti dei chatbot tradizionali. A differenza di un sistema monolitico, si basa su un'**architettura modulare** che orchestra diverse tecnologie specializzate.

Il cuore del sistema è un'architettura che integra quattro componenti tecnologici fondamentali:

1. **Large Language Model (LLM)**: Costituisce il motore per la comprensione e la generazione del linguaggio naturale, permettendo al chatbot di dialogare in modo fluido e coerente.
2. **Retrieval-Augmented Generation (RAG) e LangChain**: Per garantire risposte precise e basate su dati aziendali specifici, il sistema utilizza un'architettura RAG, implementata attraverso il framework LangChain.
 - **Cos'è la RAG?**: La Retrieval-Augmented Generation è una tecnica avanzata che migliora l'affidabilità dei Large Language Models. Invece di basarsi unicamente sulla conoscenza interna e preaddestrata del modello (che può essere datata o generica), l'approccio RAG la "aumenta" con informazioni recuperate in tempo reale da una base di conoscenza esterna. Il processo prevede che, a ogni domanda dell'utente, il sistema prima cerchi i documenti

più pertinenti nella sua base dati (fase di *Retrieval*) e poi li fornisca al LLM insieme alla domanda originale. Questo costringe il modello a basare la sua risposta sui fatti forniti, riducendo drasticamente il rischio di risposte errate o inventate ("allucinazioni") e permettendo al chatbot di utilizzare dati sempre aggiornati[4].

- **Cos'è LangChain?**: LangChain è un framework open-source che semplifica lo sviluppo di applicazioni complesse basate su LLM. Agisce come un framework di integrazione, fornendo componenti e catene (*chains*) predefinite per orchestrare flussi di lavoro complicati[3]. Nel contesto di questo progetto, LangChain è stato lo strumento utilizzato per implementare la pipeline RAG, gestendo in modo strutturato la connessione tra la base di conoscenza dei viaggi, il sistema di recupero delle informazioni e la chiamata finale all'API del LLM.
3. **Integrazione API**: Il chatbot dialoga costantemente con i sistemi legacy dell'azienda tramite API. Questo gli consente non solo di ottenere informazioni sempre aggiornate ("fresche"), ma anche di eseguire azioni concrete, come verificare la disponibilità di un tour o generare un preventivo in tempo reale.
 4. **Model Context Protocol (MCP)**: Questo componente rappresenta il vero e proprio "cuore" dell'architettura, un layer di orchestrazione semantica. A differenza della programmazione tradizionale, dove ogni scenario deve essere predefinito, l'MCP permette all'agente di comprendere autonomamente il contesto della conversazione, mantenere lo stato e scegliere dinamicamente gli strumenti più adatti in tempo reale (ad esempio, interrogare la RAG o chiamare un'API) senza bisogno di istruzioni esplicite.

Questo approccio basato sull'orchestrazione rende l'interazione più fluida ed efficace, simile a quella con un collega esperto.

Capitolo 4

Integrazione con Microsoft Teams

L'integrazione con Microsoft Teams ha rappresentato la prima e più importante fase del progetto di stage, come concordato nella proposta iniziale. L'obiettivo era collegare l'assistente conversazionale esistente, che è stato descritto nel capitolo precedente, al canale di comunicazione aziendale. Questo capitolo documenta l'intero processo, dall'analisi dei requisiti fino ai risultati ottenuti.

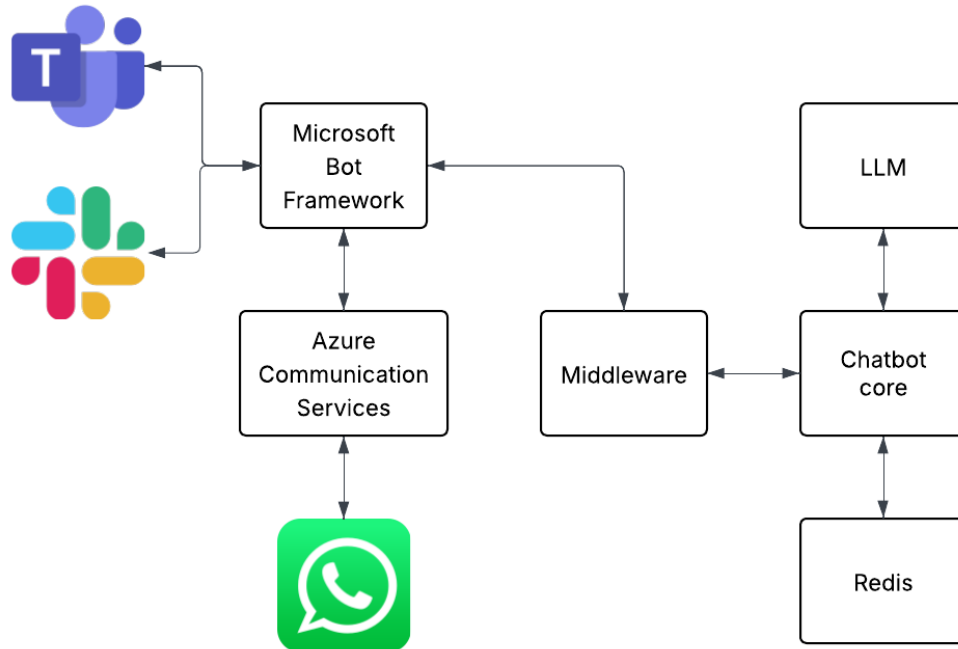


Figura 4.1: Diagramma dell'architettura della soluzione di integrazione multicanale.

4.1 Analisi dei requisiti

La fase iniziale del progetto è consistita in un'analisi approfondita per definire i requisiti funzionali e non funzionali dell'integrazione. L'obiettivo non era solo integrare il chatbot su Teams, ma creare una soluzione sicura e scalabile.

Requisiti funzionali

- **Connettività del canale:** Il requisito primario era abilitare un canale di comunicazione bidirezionale tra gli utenti su Microsoft Teams e il core del chatbot. Il middleware doveva essere in grado di ricevere i messaggi (*activity*) inviati dall'utente su Teams e di inoltrare le risposte generate dal chatbot all'utente corretto.
- **Supporto per messaggi interattivi:** L'integrazione non doveva limitarsi al solo testo. Era necessario supportare i componenti interattivi

di Teams, come le *Adaptive Cards* e i pulsanti, per replicare la user experience (es. la selezione di un viaggio) in modo efficace sull'interfaccia di Teams.

- **Autenticazione e autorizzazione:** Era cruciale implementare un meccanismo di sicurezza per validare che le richieste in ingresso provenissero effettivamente dal Microsoft Bot Framework. Allo stesso modo, le chiamate in uscita dal middleware verso le API di Microsoft dovevano essere correttamente autenticate.
- **Gestione del contesto conversazionale:** Il sistema doveva mantenere lo stato della conversazione per ogni singolo utente su Teams, permettendo al chatbot di gestire più conversazioni con più utenti alla volta.

Requisiti non funzionali

- **Integrazione architetturale:** La soluzione doveva essere sviluppata come un middleware basato su NestJS, per garantire coerenza e una facile integrazione con l'architettura tecnologica preesistente in azienda.
- **Flessibilità e controllo:** L'interazione con le API del Bot Framework doveva avvenire tramite chiamate REST dirette, evitando l'uso dell'SDK per avere un controllo maggiore sul flusso di comunicazione e sulla gestione degli errori.
- **Sicurezza:** Oltre all'autenticazione, la soluzione doveva aderire alle *best practice* di sicurezza, proteggendo dati sensibili come le credenziali (client ID, client secret) tramite un sistema di gestione della configurazione.
- **Usabilità e performance:** L'esperienza utente doveva essere fluida, con tempi di risposta bassi. L'interfaccia doveva risultare naturale per un utente abituato all'ambiente Teams.
- **Manutenibilità:** Il codice doveva essere ben documentato e modulare per facilitare aggiornamenti futuri.

4.2 Progettazione dell'integrazione

Sulla base dei requisiti definiti, è stata progettata un'architettura basata su un **middleware**, un'applicazione intermedia che agisce da orchestratore tra

Microsoft Teams e il core del chatbot aziendale. Questa scelta disaccoppia la logica di comunicazione, specifica per il canale, dalla logica di business del chatbot, garantendo maggiore manutenibilità e scalabilità futura.

La progettazione si è basata sulle seguenti decisioni architetturali chiave:

- **Scelta tecnologica (NestJS):** Per rispettare il requisito di integrazione architetturale, è stato scelto di sviluppare il middleware utilizzando il framework NestJS. Questa decisione ha garantito la coerenza con lo stack tecnologico già in uso in azienda, semplificando la gestione del progetto e la sua futura evoluzione.
- **Interazione tramite API REST dirette:** In linea con il requisito di flessibilità e controllo, si è deciso di non utilizzare l'SDK di Microsoft Bot Framework, preferendo un'interazione diretta con le API REST fornite dal servizio. Questo approccio, anche se molto più complesso da implementare, offre un controllo di più basso livello sul flusso di dati, sulla gestione degli errori e sull'autenticazione, eliminando le regole imposte dall'SDK.
- **Struttura del middleware:** È stata progettata un'applicazione modulare con responsabilità ben definite:
 - Un **Controller** dedicato a esporre un endpoint webhook, che funge da unico punto di ingresso per tutte le **activity** provenienti da Microsoft Teams (e successivamente anche dagli altri canali).
 - Un **Service Layer** per incapsulare la logica di business. Questo include un servizio specifico per Teams per gestire la validazione dei token in ingresso e l'ottenimento dei token di autenticazione per le risposte, e l'interfacciamento con i servizi del chatbot preesistente.
 - Una **Classe di interfaccia dati** per "tradurre" le risposte generiche del chatbot nel formato specifico di Teams, come le *Adaptive Cards*, permettendo così a Teams di riconoscere e interpretare al meglio il messaggio di risposta.

Questa architettura è stata ideata e creata per essere robusta e scalabile, creando una netta separazione tra la logica di canale e quella applicativa del chatbot. La fase successiva è stata quella dello sviluppo e del testing di questi componenti.

4.3 Sviluppo e test

Questa sezione descrive la fase di sviluppo del middleware, quindi traducendo le decisioni di progettazione descritte nella sezione precedente in codice funzionante. Verranno presentati e commentati i blocchi di codice più significativi, illustrando come i requisiti funzionali e non funzionali sono stati soddisfatti a livello implementativo.

4.3.1 Gestione del flusso attraverso il controller

Il cuore del middleware è rappresentato dal `TeamsChatController`, una classe NestJS che, come anticipato in fase di progettazione, agisce da webhook unificato. Espone un singolo endpoint `POST /api/chat/teams` che riceve tutte le `activity` inviate dal Microsoft Bot Framework.

L'utilizzo dei decoratori standard di NestJS, come `@Controller` e `@Post`, ha permesso di descrivere il codice e la logica in modo chiaro e preciso. L'inserimento dei decoratori è una pratica decisamente incoraggiata dagli sviluppatori senior aziendali. La classe fa molto spesso uso del pattern della *Dependency Injection* per ricevere nel suo costruttore le istanze dei servizi esterni di cui ha bisogno per funzionare, permettendo così una separazione netta delle responsabilità. I servizi iniettati sono:

- **LgChatService:** Per comunicare con il core del chatbot preesistente.
- **TeamsChatService:** Per gestire la logica specifica di Teams, come l'autenticazione.
- **HistoryService:** Per interagire con Redis, accedere alla cronologia dei messaggi e gestire lo stato della conversazione.
- **ConfigService:** Per accedere in modo sicuro alle variabili d'ambiente e alle credenziali dei servizi esterni.

Il metodo principale `handleMessage(@Body() activity: any)` riceve l'intero oggetto `activity` inviato da Teams e orchestra tutte le operazioni successive: autenticazione, gestione dello stato, inoltro al motore del chatbot e, infine, la formattazione e l'invio della risposta.

Lo snippet di codice seguente mostra la struttura del controller, evidenziando il costruttore e il metodo principale della classe.

```
// teamsChat.controller.ts

@ApiTags('TeamsChat')
```

```

@Controller('api/chat/teams')
export class TeamsChatController {
  // ...
  constructor(
    private readonly LgChatService: LgChatService,
    private readonly teamsChatService: TeamsChatService,
    private readonly configService: ConfigService,
    private readonly redisService: HistoryService,
  ) {
    // ...
  }

  @Post()
  async handleMessage(
    @Body() activity: any,
    @Res() response: ExpressResponse,
    @Headers() headers: any,
  ) {
    // Logica di orchestrazione...
  }
  // ...
}

```

Questo metodo di centralizzare il controller permette di avere un unico punto di controllo per la gestione delle richieste provenienti da Teams, semplificando il debug e la scalabilità futura del sistema.

4.3.2 Gestione della sicurezza e dell'autenticazione

Un requisito non funzionale di primaria importanza era garantire la sicurezza dell'endpoint esposto dal codice. Per soddisfare questo requisito, è necessario assicurarsi che ogni richiesta ricevuta provenga effettivamente dal Microsoft Bot Framework.

Questo è stato ottenuto implementando un meccanismo di *authentication* basato su JSON Web Token (JWT). Ogni *activity* inviata dal servizio di Microsoft è accompagnata da un header *Authorization* contenente un token *Bearer* firmato. Il middleware ha il compito di validare questo token prima di processare qualsiasi richiesta.

Per automatizzare questo processo in modo pulito e riutilizzabile, è stata creata una *Guard* custom di NestJS, la *JwtAuthGuard*. Questa guard intercetta ogni chiamata in arrivo all'endpoint e ne esegue la validazione, seguen-

do lo standard di sicurezza OpenID. Il processo di validazione implementato nella funzione `verifyToken` viene eseguito seguendo questa sequenza:

1. **Estrazione del token:** Il token JWT viene estratto dall'header con campo `Authorization` della richiesta.
2. **Recupero della chiave pubblica:** Il token è firmato con una chiave privata di Microsoft. Per verificarne l'autenticità, è necessario recuperare la chiave pubblica corrispondente. La guard ottiene dinamicamente l'URI del JWKS (JSON Web Key Set) di Microsoft, un endpoint pubblico che contiene l'elenco delle chiavi utilizzate per la firma.
3. **Decodifica e verifica:** La guard decodifica l'header del token per estrarre l'ID della chiave (`kid`), usa questo ID per trovare la chiave pubblica corretta nel JWKS e infine utilizza una libreria (`jsonwebtoken`) per verificare la firma del token con l'algoritmo RS256.

Se la firma è valida, la richiesta viene inoltrata al controller; in caso contrario, la guard solleva un'eccezione `401 Unauthorized` che blocca immediatamente la richiesta e evita di proseguire con la logica del controller [6].

Lo snippet di codice seguente mostra la logica di validazione implementata.

```
// jwt-auth.guard.ts

async verifyToken(token: string, uri: Promise<string>):
  ↳ Promise<any> {
  const resolvedUri = await uri;
  let jwksClientTeams = jwks({
    jwksUri: resolvedUri,
  });

  try {
    // Decodifica l'header del token per ottenere il kid
    const decodedHeader = jwt.decode(token, { complete: true
    ↳ })?.header;
    if (!decodedHeader) {
      throw new UnauthorizedException('Invalid token');
    }

    // Ottieni la chiave pubblica dal JWKS usando il kid
    const key = await
    ↳ jwksClientTeams.getSigningKey(decodedHeader.kid);
    const publicKey = key.getPublicKey();
```

```
// Verifica la firma del token con la chiave pubblica
return jwt.verify(token, publicKey, { algorithms: ['RS256'] });
} catch (error) {
  console.error('Token validation error:', error);
  throw new UnauthorizedException('Token validation failed');
}
}
```

Questo sistema garantisce che soltanto le richieste autenticate e provenienti da fonti riconosciute (come Microsoft) possano interagire con il chatbot, in modo tale da proteggere il sistema da accessi non autorizzati e potenzialmente pericolosi.

4.3.3 Gestione dello stato conversazionale con Redis

Per implementare il requisito di gestire dialoghi complessi e multi-turno, era essenziale che il middleware fosse in grado di mantenere lo stato della conversazione. Un chatbot *stateless* tratterebbe ogni messaggio come un evento isolato, rendendo quindi impossibile gestire flussi articolati come la prenotazione di un viaggio.

La soluzione adottata è stata quella di mantenere traccia della cronologia di ogni conversazione in un database Redis.

Per poter associare una cronologia a una specifica conversazione, per ogni messaggio in ingresso viene generato un identificatore di sessione univoco, il `sessionId`. Questo identificatore è la chiave primaria con cui i dati vengono salvati e recuperati da Redis. La sua struttura, visibile nello snippet di codice, è ottenuta concatenando due campi forniti dall'oggetto `activity` di Teams: l'ID dell'utente (`activity.from.id`) e l'ID della conversazione (`activity.conversation.id`).

```
// Estrazione dal teamsChat.controller.ts
const sessionId = activity.from.id + '@' + activity.conversation.id;
```

Questa scelta garantisce che la cronologia di un utente in una chat privata sia mantenuta separata dalla sua cronologia in un canale di gruppo, permettendo al bot di gestire diversi canali di conversazione per lo stesso utente senza sovrapposizioni.

L'interazione con Redis, già utilizzata nell'architettura del chatbot, è stata istanziata all'interno dell'`HistoryService`. Questo servizio sfrutta la libreria `@langchain/community` e in particolare la classe `RedisChatMessageHistory`. Ogni conversazione viene salvata usando un `sessionId` univoco come chiave

e viene impostato un TTL (Time-To-Live) per far scadere automaticamente le sessioni inattive, ottimizzando l'uso della memoria.

Una funzionalità chiave sviluppata in questa fase è stata quella di dare all'utente il controllo esplicito sulla propria sessione. Per fare ciò, è stata implementata la funzione `clearSessionHistory`. Questo metodo, come mostrato nello snippet di codice, si occupa di eliminare da Redis sia la cronologia dei messaggi sia i dettagli del viaggio associati a una specifica sessione.

```
// history.service.ts

async clearSessionHistory(sessionId: string): Promise<void> {
  const historyInstance = this.getHistoryInstance(sessionId);
  console.log(`Clearing history for session: ${sessionId}`);

  // Cancella i dettagli del viaggio associati
  await this.redisClient.del(`trip_details:${sessionId}`);

  // Cancella la cronologia dei messaggi
  // usando l'istanza di LangChain
  return await historyInstance.clear();
}
```

Questa funzione viene invocata dal `TeamsChatController` quando riceve un'attività di tipo `invoke` da Microsoft Teams, generata dalla pressione del pulsante "New Chat" da parte dell'utente.

4.3.4 Creazione di risposte interattive per Teams

Un requisito funzionale chiave del progetto era "mappare" i flussi conversazionali esistenti sull'interfaccia di Microsoft Teams. Questo significava tradurre le risposte e le domande del chatbot in componenti visivi e interattivi, sfruttando le possibilità di modifica del messaggio offerta dalla piattaforma per migliorare l'esperienza utente.

Per raggiungere questo importante obiettivo, è stata creata una classe dedicata, `TeamsResponse`. Questa classe agisce come un *builder*, un costruttore che gestisce il processo della creazione dei payload JSON richiesti da Teams per visualizzare messaggi interattivi, come le Hero Cards e le Adaptive Cards.

Il `TeamsChatController`, a seconda dello stadio della conversazione restituito dal motore del chatbot, chiama un metodo specifico preso dalla classe `TeamsResponse` per costruire la risposta adeguata. Per esempio, quando il chatbot deve proporre all'utente una lista di viaggi, viene utilizzato il metodo `setCardMessage`.

Questo metodo genera dinamicamente una sequenza di Card che contengono immagini, come specificato dal template del messaggio "Hero Cards", una per ogni pacchetto di viaggio. Ogni card è costruita per contenere le informazioni di base del tour (nome, immagine se disponibile) e un pulsante interattivo "Scopri di più" che rimanda alla pagina web del tour attraverso un url. Il seguente snippet di codice, preso dalla classe `TeamsResponse`, mostra la logica per la creazione di una singola Hero Card all'interno del carosello.

```
// Metodo semplificato da TeamsResponse.ts
// ...
this.body.attachments.push({
  contentType: 'application/vnd.microsoft.card.hero',
  content: {
    text: tour.metadata.name, // Nome del tour
    images: [{ url: tour.metadata.imageName }], // Immagine del tour
    buttons: [
      {
        type: 'openUrl', // Tipo di azione: apre un link
        title: 'Scopri di più',
        value: tour.metadata.source, // URL della pagina del tour
      },
    ],
  },
});
// ...
```

Oltre alle Hero Cards, sono state utilizzate anche le **Adaptive Cards**. Ad esempio, il metodo `setNewChatMessage` genera una card con il messaggio di fine conversazione e un pulsante "New Chat". Questo pulsante è configurato con un'azione di tipo `Action.Execute`, che, quando premuto, invia al middleware un payload `invoke` che attiva la logica di eliminazione della cronologia descritta nella sezione precedente.

L'utilizzo di una classe dedicata per la costruzione delle risposte ha permesso di mantenere il codice del controller pulito e concentrato chiaramente sulla logica del flusso, delegando tutta la logica di presentazione a un componente specifico facilmente modificabile.

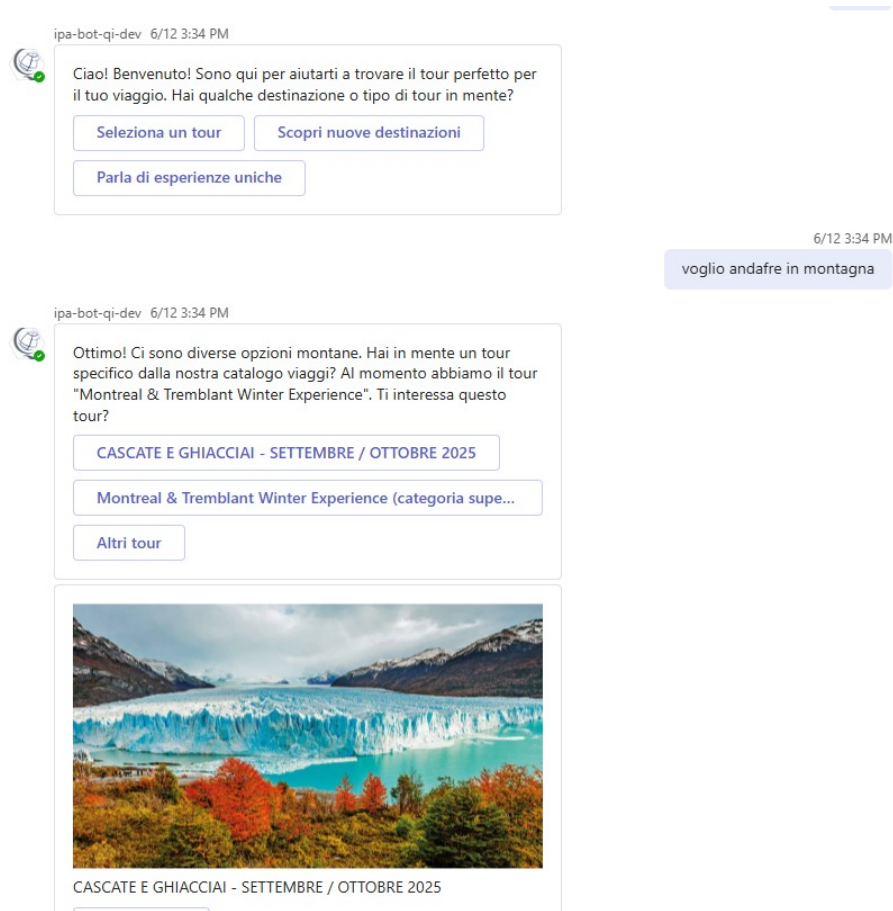


Figura 4.2: Esempio di conversazione su Microsoft Teams. Si noti la risposta del chatbot che include pulsanti di azione e una card interattiva con immagine.

4.3.5 Processo di testing

Per garantire che il middleware funzionasse come dovuto e per correggere eventuali errori, è stata adottata una strategia di testing, che ha compreso test unitari per i singoli componenti e, soprattutto, test funzionali e di integrazione per l'intero flusso di comunicazione.

Test unitari

A livello di codice, sono stati predisposti test unitari utilizzando il framework **Jest**, integrato di default in NestJS. Sono stati creati file di specifica (`.spec.ts`) per i componenti principali, come il `TeamsChatController` e il

TeamsChatService. Questi test avevano l'obiettivo di verificare la corretta istanziamento dei componenti e la loro capacità di interagire con il sistema di dependency injection di NestJS. Assicurando così l'assenza di errori a livello di configurazione del modulo.

Test funzionali e di integrazione

Il test più significativo è stato quello funzionale, volto a verificare l'intero flusso di comunicazione tra un client di messaggistica e il middleware. Per questa attività, è stato utilizzato uno strumento fondamentale fornito da Microsoft: il **Bot Framework Emulator**.

Il processo di testing con l'emulatore si è svolto nel seguente modo:

1. L'applicazione middleware NestJS veniva eseguita in locale sul computer di sviluppo, rimanendo aperta e in ascolto sulla porta di default (`localhost:3000`).
2. Tramite un servizio di tunneling chiamato **ngrok**, l'endpoint locale del middleware veniva esposto su un indirizzo internet pubblico e sicuro (HTTPS), rendendolo così raggiungibile dai servizi cloud di Microsoft.
3. Il Bot Framework Emulator veniva quindi configurato per comunicare con questo indirizzo pubblico.

Questo setup ha permesso di simulare l'ambiente reale e si è rivelato fondamentale per il debug e la validazione dell'intero sistema. Attraverso l'interfaccia dell'emulatore è stato possibile:

- Inviare messaggi di testo e, soprattutto, **activity** di tipo **invoke** per simulare la pressione dei pulsanti sulle Adaptive Cards e testare le funzionalità interattive.
- Ispezionare in dettaglio sia la richiesta JSON inviata al middleware, sia la risposta ricevuta, analizzando la struttura delle card e verificandone la corretta formattazione.

L'utilizzo del Bot Framework Emulator ha quindi permesso di eseguire il debug del codice in modo rapido ed efficiente, senza la necessità di effettuare un deploy dell'applicazione ad ogni modifica.

4.4 Risultati ottenuti

L'attività di sviluppo e testing ha portato al completamento con successo dell'integrazione del chatbot aziendale sulla piattaforma Microsoft Teams,

raggiungendo tutti gli obiettivi primari definiti nella fase di analisi dei requisiti.

I principali risultati ottenuti possono essere definiti come segue:

- **Realizzazione di un middleware funzionante:** È stato sviluppato e testato un middleware robusto e scalabile, basato su **NestJS**, che agisce come un ponte efficace tra il core del chatbot e l'API di Microsoft Bot Framework.
- **Piena accessibilità del chatbot su Teams:** L'assistente conversazionale è ora pienamente accessibile e utilizzabile dagli utenti finali (dipendenti o clienti) direttamente all'interno di Microsoft Teams. La comunicazione è bidirezionale e supporta conversazioni complesse e multi-turno grazie alla gestione dello stato tramite Redis.
- **Esperienza utente ricca e integrata:** L'integrazione supporta pienamente i messaggi interattivi, come le *Adaptive Cards* e i pulsanti. Questo ha permesso di replicare i flussi conversazionali esistenti in modo intuitivo, offrendo un'esperienza utente fluida e nativa per la piattaforma.
- **Sicurezza del canale garantita:** Il meccanismo di autenticazione basato su token **JWT** ha permesso di proteggere l'endpoint del middleware, assicurando che solo le richieste legittime provenienti da Microsoft vengano processate.

Il superamento positivo della fase di test funzionale con il Bot Framework Emulator ha confermato la fattibilità tecnica e la stabilità della soluzione sviluppata, di conseguenza validando le scelte architetturali e tecnologiche fatte in fase di progettazione.

Completati i test tecnici, la soluzione è stata sottoposta a una fase di test di usabilità con un gruppo di colleghi di Par-Tec. Il riscontro è stato positivo: i tester hanno particolarmente apprezzato l'uso dei messaggi interattivi, giudicati efficaci nel rendere l'esperienza utente più intuitiva e visivamente chiara. Durante questa validazione non sono emerse criticità significative il che conferma il raggiungimento degli obiettivi di usabilità del progetto.

Capitolo 5

Integrazione con Slack

Questo capitolo descrive l'integrazione dell'assistente conversazionale con un secondo canale di messaggistica aziendale: Slack. A differenza del lavoro svolto per Microsoft Teams, questa integrazione non ha richiesto lo sviluppo di nuovo codice, ma si è basata su un processo di configurazione.

L'obiettivo di questo capitolo è duplice: documentare i passaggi tecnici necessari per abilitare il canale Slack e, soprattutto, rendere questo processo una dimostrazione pratica dell'utilità architetturale del middleware. Il successo di questa seconda integrazione, ottenuta scrivendo alcuna riga di codice, dimostra, la scalabilità e la funzione multicanale della soluzione progettata.

5.1 Il ruolo del Microsoft Bot Framework

La ragione per cui è stato possibile integrare Slack senza modifiche al codice del middleware risiede nel ruolo di **astrazione** svolto dal Microsoft Bot Framework. Senza un tale livello di astrazione, sarebbe stato necessario sviluppare un connettore specifico per le API di Slack, capace di interpretarne i formati di messaggio e i meccanismi di autenticazione, duplicando di fatto gran parte del lavoro già svolto per Teams.

Il Bot Framework, invece, agisce come un intermediario. Il suo funzionamento può essere schematizzato come segue:

- **Comunicazione in ingresso:** Quando un utente invia un messaggio su un canale Slack collegato, i servizi di Microsoft intercettano il payload specifico di Slack e lo trasformano in un oggetto JSON standardizzato, l'oggetto **activity**. Il middleware riceve quindi sempre lo

stesso formato di dati, indipendentemente dal fatto che il messaggio provenga da Teams o da Slack.

- **Comunicazione in uscita:** L'astrazione funziona anche nella direzione opposta. Quando il middleware invia una risposta, come una *Adaptive Card*, la invia al Bot Framework utilizzando un formato standard. È poi compito del Bot Framework tradurre questa card nel formato corretto per essere visualizzata correttamente all'interno dell'interfaccia di Slack.

È proprio questo meccanismo di astrazione che rende la soluzione scalabile. Il middleware deve conoscere e gestire un solo protocollo di comunicazione (quello del Bot Framework), mentre la complessità della gestione dei singoli canali è delegata interamente alla piattaforma Microsoft.

5.2 Configurazione del canale Slack

Anche se non sono state necessarie modifiche al codice, l'abilitazione del canale Slack ha richiesto un accurato processo di configurazione per collegare tre diverse piattaforme: l'ambiente di sviluppo di Slack, il portale di Microsoft Azure e il middleware stesso.

Il processo si è articolato nei seguenti passaggi:

1. **Creazione dell'applicazione su Slack:** Il primo passo è stato accedere al portale degli sviluppatori di Slack (api.slack.com) e creare una nuova "Slack App". Questa applicazione rappresenta il bot all'interno dell'applicazione di Slack.
2. **Configurazione dei permessi (scopes):** All'interno delle impostazioni dell'app, sono stati configurati gli "scope" OAuth necessari al bot per operare correttamente. Gli scope definiscono le autorizzazioni che l'app richiede, come ad esempio la capacità di leggere i messaggi nei canali (`channels:history`), inviare messaggi (`chat:write`) e visualizzare informazioni sugli utenti (`users:read`).
3. **Ottenimento delle credenziali:** Una volta configurata l'app e installata in uno spazio di lavoro di test, il portale di Slack ha fornito le credenziali necessarie per il collegamento con servizi esterni: il `Client ID`, il `Client Secret` e il `Signing Secret`.

4. **Collegamento di Slack:** Successivamente, navigando all'interno del portale di Microsoft Azure, è stata aperta la risorsa "Azure Bot" precedentemente creata per Teams. All'interno della sezione "Canali" (Channels) della risorsa, è stato aggiunto il canale "Slack". Il portale di Azure ha quindi richiesto l'inserimento delle credenziali ottenute al punto 3. Una volta inserite e salvate, il collegamento tra Azure e l'app Slack è stato stabilito.

Completato questo processo, il Bot Framework ha iniziato a ricevere i messaggi provenienti dallo spazio di lavoro Slack collegato e a inoltrarli, sotto forma di **activity** standardizzate, all'endpoint del middleware, rendendo l'assistente immediatamente operativo anche su questo nuovo canale.

5.3 Risultati e validazione architetturale

Una volta completato il collegamento tra l'app Slack e la risorsa Azure Bot, l'assistente conversazionale è diventato pienamente operativo sul nuovo canale senza che fosse necessaria alcuna modifica o ridistribuzione (*deploy*) del codice del middleware.

I test condotti sullo spazio di lavoro Slack hanno confermato che tutte le funzionalità già implementate per Microsoft Teams erano presenti e funzionanti, inclusa la gestione dello stato delle conversazioni e il rendering dei messaggi interattivi tramite *Adaptive Cards*.

Questo risultato, sebbene ottenuto senza scrivere nuovo codice, è uno dei più significativi dell'intero progetto di tesi, in quanto rappresenta una validazione importante delle scelte architetturali fatte in fase di progettazione. L'aver disaccoppiato la logica del chatbot da quella dei canali, interfacciandosi esclusivamente con il livello di astrazione offerto dal Microsoft Bot Framework, si è dimostrata una strategia vincente.

Il successo dell'integrazione con Slack dimostra che il sistema è realmente **multicanale e scalabile**. L'aggiunta di futuri canali di messaggistica supportati dal Bot Framework (come ad esempio Telegram o Facebook Messenger) non richiederà uno sforzo di sviluppo, ma seguirà lo stesso processo di configurazione a basso impatto descritto in questo capitolo. È stata così provata la robustezza e flessibilità dell'architettura del middleware.

Capitolo 6

Integrazione con WhatsApp Business

L'integrazione con WhatsApp Business ha rappresentato l'ultima fase implementativa del progetto, nonché la più complicata dal punto di vista tecnico. A differenza di Slack, l'aggiunta del canale WhatsApp non è stata un'operazione di sola configurazione, ma ha richiesto un'analisi e uno sviluppo dedicati per conoscere le specifiche della piattaforma e la complessità dei suoi meccanismi di autenticazione.

Questo capitolo documenta il processo di integrazione, partendo dalla scelta della piattaforma Azure utilizzata come "ponte" verso le API di WhatsApp, per poi approfondire le sfide di sviluppo affrontate e le soluzioni implementate per ottenere un'integrazione funzionante.

6.1 Analisi e scelta della piattaforma

A differenza di Microsoft Teams e Slack, WhatsApp Business non è un canale supportato nativamente dal Microsoft Bot Framework. Quindi, l'integrazione con questo canale ha richiesto l'utilizzo di un servizio intermedio che fungesse da "gateway" o "connettore" tra il middleware e le API di WhatsApp. Come previsto dalla fase di studio di fattibilità del progetto, sono state considerate diverse opzioni, tra cui l'utilizzo di provider esterni specializzati come Twilio o l'interfacciamento diretto con la Cloud API di Meta.

La scelta finale, tuttavia, è ricaduta sull'utilizzo di **Azure Communication Services (ACS)**, una piattaforma di comunicazione offerta da Micro-

soft stessa. Questa decisione è stata guidata da due principali motivazioni di sviluppo:

1. **Mantenimento dell'ecosistema Azure:** La scelta di ACS ha permesso di mantenere l'intera infrastruttura del bot all'interno di un unico ecosistema cloud, quello di Microsoft Azure. Questo approccio semplifica la gestione, la scalabilità e la sicurezza, sfruttando gli stessi meccanismi di autenticazione e monitoraggio già in uso per il resto dei servizi.
2. **Semplicità di sviluppo del middleware:** L'obiettivo era creare un controller unificato, il `TeamsChatController`, che gestisse tutte le comunicazioni in ingresso provenienti da servizi Azure. Poiché sia il Bot Framework (per Teams e Slack) sia Azure Communication Services (per WhatsApp) sono servizi Azure, è stato possibile instradare gli eventi di entrambi verso lo stesso endpoint. Questa scelta ha evitato la necessità di creare un controller separato e una logica di routing dedicata per WhatsApp, portando a un'architettura del middleware più semplice e manutenibile.

Anche se questa decisione abbia semplificato la struttura del controller, l'implementazione pratica della comunicazione con Azure Communication Services si è dimostrata insidiosa, in particolare per quanto riguarda l'autenticazione delle chiamate API in uscita, che verranno analizzate nella sezione seguente.

6.2 Sfide di integrazione e sviluppo

Come anticipato nella sezione 2.2, l'integrazione con WhatsApp Business, pur utilizzando i servizi Azure, ha presentato sfide tecniche notevolmente superiori a quelle incontrate con i canali nativi del Bot Framework. La documentazione frammentaria e la tipologia di connessione indiretta hanno richiesto un approccio basato su svariate prove e reverse engineering di alcuni meccanismi.

Gestione dei messaggi in ingresso

La prima differenza sostanziale riguarda il formato dei messaggi in ingresso. A differenza di Teams e Slack, che inviano al middleware un oggetto `activity` standard, i messaggi da WhatsApp tramite Azure Communication Services (ACS) arrivano con un tipo di evento differente chiamato `Microsoft.Communication.AdvancedMessageReceived`.

È stato quindi necessario modificare la logica del `TeamsChatController` per gestire questo caso specifico. Le informazioni cruciali, come l'ID del mittente e il contenuto del messaggio, dovevano essere estratte da una struttura dati diversa, come mostrato nel seguente frammento di codice.

```
// Frammento dal TeamsChatController
if (
  Array.isArray(activity) &&
  activity[0].eventType ===
    → 'Microsoft.Communication.AdvancedMessageReceived'
) {
  activity = activity[0]; // Estrae l'evento principale

  // Estrae l'ID utente e il messaggio da una struttura dati
  → differente
  const sessionId = activity.data.from + '@' +
    → activity.subject.split('recipient/')[1];
  bookingDetails.message = activity.data.content;

  // ...chiama il LgChatService...
}
```

Il problema dell'autenticazione in uscita

La complessità maggiore è emersa nella gestione delle risposte. Per inviare un messaggio di ritorno all'utente su WhatsApp tramite ACS, non era sufficiente un semplice token `Bearer`. L'API di ACS richiede che ogni singola richiesta sia firmata con un'autenticazione personalizzata di tipo **HMAC-SHA256**. Come sottolineato nel capitolo 2.2, la documentazione ufficiale su come generare questa firma si è rivelata scarsa e mancante di esempi. È stato quindi necessario un lungo processo di trial and error per ricostruire manualmente la logica corretta. Questo ha comportato la creazione di una funzione dedicata, `AuthenticationScript`, che esegue i seguenti passaggi per ogni messaggio in uscita:

1. Crea una stringa univoca (*string-to-sign*) concatenando l'host, la data in formato UTC, il metodo HTTP, il percorso dell'URL e un hash SHA256 del corpo della richiesta.
2. Decodifica la chiave di accesso del servizio ACS da Base64.
3. Utilizza la chiave decodificata per firmare la *string-to-sign* con l'algoritmo HMAC-SHA256.

4. Inserisce la firma risultante nell'header `Authorization` della richiesta.

Questa complicata implementazione manuale, è stata fondamentale per abilitare la comunicazione in uscita verso WhatsApp.

Limitazioni dell'interfaccia utente

Infine, un'ultima sfida riguarda le limitazioni intrinseche del canale WhatsApp rispetto a Teams. Le funzionalità interattive sono più restrittive e non sono supportate direttamente le *Adaptive Cards*. La comunicazione si basa principalmente su testo, media e pulsanti di risposta rapida, che richiedono l'approvazione dei template di messaggio da parte di Meta. Per questo progetto, ci si è concentrati sull'abilitazione della comunicazione testuale, lasciando l'esplorazione di interfacce grafiche avanzate come possibile sviluppo futuro.

6.3 Risultati ottenuti

Nonostante le significative sfide tecniche incontrate, l'integrazione con il canale WhatsApp Business è stata completata con successo, dimostrando la flessibilità dell'architettura del middleware anche in scenari di integrazione complessi e non standard.

I principali risultati di questa fase sono i seguenti:

- **Connessione funzionante con WhatsApp:** È stata stabilita una comunicazione bidirezionale tra il chatbot e gli utenti finali su WhatsApp, utilizzando Azure Communication Services come gateway. Il middleware è in grado di ricevere correttamente i messaggi in ingresso e di inviare le risposte testuali generate dal chatbot.
- **Superamento delle sfide di autenticazione:** La sfida più complessa, ovvero l'implementazione manuale del meccanismo di autenticazione basato su firma HMAC-SHA256, è stata superata con successo. Questo ha permesso al middleware di inviare messaggi in uscita in modo sicuro e autorizzato. La funzione `AuthenticationScript`, mostrata di seguito, è stata sviluppata appositamente per generare la firma richiesta a ogni chiamata.

```
// Funzione per l'autenticazione delle chiamate a WhatsApp
↳ via ACS
function AuthenticationScript(endpoint, key, body, url) {
  const dateStr = new Date().toUTCString();
```

```

const hashedBodyStr =
  ↪ CryptoJS.SHA256(JSON.stringify(body))
    .toString(CryptoJS.enc.Base64);
const hostStr = endpoint.replace('https://', '');
url = url.toString().replace(`${endpoint}`, '');

const stringToSign = 'POST\n' + url + '\n' + dateStr +
  ↪ ';'
    + hostStr + ';' + hashedBodyStr;

const encKey = CryptoJS.enc.Base64.parse(key);
const signature = CryptoJS.HmacSHA256(stringToSign,
  ↪ encKey)
    .toString(CryptoJS.enc.Base64);

const headers = {
  'Date': dateStr,
  'x-ms-content-sha256': hashedBodyStr,
  'Authorization': `HMAC-SHA256
  ↪ SignedHeaders=date;host;x-ms-content-sha256&
  Signature=${signature}`
};

return { method: 'POST', headers: headers, body:
  ↪ JSON.stringify(body) };
}

```

- **Validazione dell’approccio architetturale:** Il successo di questa integrazione ha dimostrato la scelta di un’architettura a middleware come appropriata. Ha provato che, con gli adattamenti necessari, il sistema è in grado di interfacciarsi non solo con i canali nativi del Bot Framework, ma anche con servizi di terze parti.

Va notato che, a causa delle limitazioni intrinseche del canale, l’integrazione si è concentrata sull’implementazione dei flussi conversazionali basati su testo. Il risultato ottenuto è quindi un’importante prova di concetto (*Proof of Concept*) che conferma la capacità dell’architettura di adattarsi a contesti tecnologici diversi.

Capitolo 7

Conclusioni

Questo capitolo conclusivo ha l'obiettivo di tirare le somme del lavoro di tesi svolto, analizzando i risultati finali del progetto, il valore che ha generato per l'azienda e le competenze acquisite durante il percorso di stage. Infine, verranno delineate le possibili evoluzioni e gli sviluppi futuri del sistema di integrazione multicanale realizzato.

7.1 Sintesi dei risultati raggiunti

Il progetto di stage si è concluso con il raggiungimento di tutti gli obiettivi prefissati, portando risultati concreti e validando la fattibilità tecnica dell'approccio proposto. Il lavoro svolto ha permesso di trasformare un'assistente conversazionale, inizialmente limitato a un'unica interfaccia, in una piattaforma di comunicazione multicanale.

I risultati chiave possono essere così sintetizzati:

- **Sviluppo di un middleware scalabile:** Il risultato principale è la progettazione e l'implementazione di un middleware robusto, disaccoppiato e scalabile, basato su NestJS. Questo layer intermedio si è dimostrato efficace nel gestire la comunicazione tra il core del chatbot e diversi canali esterni, agendo come un orchestratore centrale.
- **Integrazione completa con Microsoft Teams:** È stata completata con successo l'integrazione con il canale prioritario, Microsoft Teams. La soluzione supporta oltre la comunicazione testuale, anche un'esperienza utente ricca e interattiva tramite l'uso di *Adaptive Cards*, con meccanismi di sicurezza e gestione dello stato pienamente funzionanti.
- **Validazione architetturale con Slack:** L'integrazione con Slack, ottenuta tramite sola configurazione e senza modifiche al codice, ha

fornito una validazione della scalabilità dell'architettura. Questo ha dimostrato che il design del middleware è effettivamente multicanale, come teorizzato in fase di progettazione.

- **Prova di concetto funzionante per WhatsApp:** Nonostante le notevoli sfide tecniche, è stata realizzata un'integrazione funzionante con WhatsApp Business tramite Azure Communication Services. Questo risultato ha confermato la flessibilità del middleware, dimostrandone la capacità di interfacciarsi anche con canali non nativi del Bot Framework che richiedono logiche di autenticazione complesse e personalizzate.

7.2 Valore aggiunto per l'azienda

Oltre ai risultati tecnici, il progetto ha generato un significativo valore aggiunto per l'azienda, Par-Tec S.p.A., in linea con gli obiettivi strategici delineati all'inizio dello stage. Il middleware sviluppato non è un semplice esercizio accademico, ma una tecnologia che ha il potenziale di portare benefici concreti all'azienda.

- **Miglioramento dell'accessibilità e adozione:** Integrando il chatbot aziendale all'interno di piattaforme di messaggistica come Microsoft Teams, Slack e WhatsApp, che sono già utilizzate quotidianamente da dipendenti e clienti, è stata notevolmente migliorata l'accessibilità ai servizi aziendali. Questa familiarità d'uso migliora l'adozione dello strumento, che massimizza così il ritorno sull'investimento della tecnologia di intelligenza artificiale creata in precedenza dall'azienda.
- **Aumento dell'efficienza operativa:** La disponibilità del chatbot su questi nuovi canali contribuisce a ridurre i tempi di risposta alle richieste degli utenti, sia interni che esterni. L'automazione delle risposte a domande frequenti e la gestione di flussi standardizzati permettono di alleggerire il carico di lavoro del personale umano, che può dedicarsi ad attività alternative, a maggior valore aggiunto.
- **Creazione di una tecnologia strategica per il futuro:** Il lavoro svolto ha fornito a Par-Tec una base solida per future integrazioni multicanale. Il middleware è stato progettato per essere scalabile, e come dimostrato dall'integrazione con Slack, l'aggiunta di nuovi canali supportati dal Bot Framework richiederà uno sforzo minimo. Inoltre, lo studio e l'implementazione del prototipo per WhatsApp hanno fornito all'azienda dati concreti sulle metodologie e sui costi di tale canale, permettendo di prendere decisioni di investimento future più informate.

7.3 Competenze acquisite durante lo stage

L'esperienza di stage curriculare in Par-Tec ha rappresentato un'importante occasione di crescita, permettendomi di applicare le conoscenze teoriche acquisite nel percorso universitario a un progetto software complesso e innovativo. Le attività svolte mi hanno permesso di sviluppare e migliorare certe competenze, sia tecniche che metodologiche.

- **Progettazione e sviluppo di sistemi conversazionali:** Ho acquisito una comprensione pratica dell'architettura di un sistema conversazionale avanzato, basato su LLM e pattern come RAG e MCP. Ho imparato a progettare e sviluppare un middleware scalabile, con una chiara separazione delle responsabilità tra logica di business e logica di integrazione dei canali.
- **Integrazione di API e piattaforme cloud:** Il progetto mi ha dato una importante esperienza pratica nell'integrazione con API RESTful complesse, in particolare quelle dell'ecosistema Microsoft. Ho imparato a gestire i flussi di autenticazione di Microsoft Teams (OAuth 2.0, JWT) e l'autenticazione di Azure Communication Services (HMAC-SHA256).
- **Sviluppo backend con NestJS:** Ho consolidato le mie capacità di sviluppo backend utilizzando il framework NestJS e il linguaggio TypeScript. Ho applicato principi di programmazione solida come la Dependency Injection per creare un'applicazione modulare e testabile.
- **Sicurezza delle applicazioni:** Ho affrontato direttamente il tema della sicurezza nelle applicazioni web, implementando meccanismi di validazione dei token e gestendo in modo sicuro le credenziali e le chiavi API tramite servizi di configurazione.
- **Metodologie di testing:** Ho applicato una strategia di testing a più livelli, scrivendo test unitari con Jest e, soprattutto, conducendo test funzionali e di integrazione con strumenti specifici come il Bot Framework Emulator e ngrok.
- **Problem-solving in contesti reali:** Una delle competenze più preziose acquisite è stata la capacità di affrontare e risolvere problemi tecnici in presenza di documentazione incompleta o poco chiara, in particolare durante l'integrazione con WhatsApp. Questo ha richiesto un approccio di sperimentazione, debug e reverse engineering.

7.4 Prospettive future e possibili sviluppi

Il lavoro di tesi svolto durante lo stage ha posto solide fondamenta per l'evoluzione multicanale dell'assistente conversazionale. La flessibilità dell'architettura del middleware apre la strada a numerosi sviluppi futuri, volti a potenziare ulteriormente le funzionalità del sistema e ad estenderne la portata e l'utilizzo.

- **Estensione a nuovi canali di messaggistica:** La validazione architetturale ottenuta con l'integrazione di Slack ha dimostrato che il sistema è pronto per essere esteso ad altri canali supportati dal Microsoft Bot Framework, come ad esempio Telegram o Facebook Messenger. L'aggiunta di questi canali seguirebbe un processo di sola configurazione, simile a quello descritto per Slack.
- **Potenziamento dell'integrazione con WhatsApp:** L'integrazione con WhatsApp è attualmente una prova di concetto funzionante, basata sulla comunicazione testuale. Uno sviluppo futuro sarebbe quello di migliorare l'esperienza utente su questo canale, implementando il supporto per i messaggi interattivi come i pulsanti di risposta rapida (*Quick Replies*) e i template di messaggio pre-approvati. Questo trasformerebbe il prototipo in una soluzione pienamente operativa.
- **Integrazione del canale vocale e "Zero UI":** La prospettiva futura più ambiziosa, già in fase di studio da parte dell'azienda, è l'integrazione della voce come canale di input. Questo comporterebbe l'integrazione di servizi di *speech-to-text* per interpretare i comandi vocali degli utenti. L'obiettivo a lungo termine è evolvere verso un'esperienza "Zero UI", in cui l'utente può dialogare e impartire comandi all'assistente parlando in linguaggio naturale, e l'assistente agisce di conseguenza, quasi come un collega umano.
- **Evoluzione del middleware:** Il middleware stesso potrebbe essere potenziato. Si potrebbe implementare una dashboard di monitoraggio per analizzare le performance e i volumi di traffico sui diversi canali, o addirittura un pannello di amministrazione per gestire le configurazioni dei canali in modo più visuale.

Bibliografia

- [1] Oracle Corporation. Cos'è un chatbot? <https://www.oracle.com/it/chatbots/what-is-a-chatbot/>, 2020.
- [2] IBM. Cos'è un chatbot? <https://www.ibm.com/think/topics/chatbots>, 2021.
- [3] LangChain. What is langchain? <https://github.com/langchain-ai/langchainjs>, 2025.
- [4] Medium. What is retrieval-augmented generation(rag) in llm and how it works? <https://medium.com/@sahin.samia/what-is-retrieval-augmented-generation-rag-in-llm-and-how-it-works-a8c79e35a172>, 2024.
- [5] Microsoft. Bot framework sdk. <https://github.com/microsoft/botframework-sdk/>, 2021.
- [6] Microsoft. Microsoft bot framework authentication via rest api. <https://learn.microsoft.com/en-us/azure/bot-service/rest-api/bot-framework-rest-connector-authentication?view=azure-bot-service-4.0&tabs=multitenant>, 2024.
- [7] Par-Tec. Chi siamo? <https://www.par-tec.it/il-nostro-gruppo/>, 2025.